Análise de Recursos, Timing, Desempenho, Gargalos, Limitações e Melhorias

Este texto resume os resultados da compilação Quartus Prime 25.1 (Lite) do projeto PBL02_AFJ (HPS + coprocessador gráfico), a interpretação de timing/desempenho e as principais limitações arquiteturais. Os números vêm de output_files/soc_system.fit.summary, soc_system.map.summary e soc_system.sta.summary.

1. Análise de recursos

1.1 Utilização global (Fitter)

Recurso	Usado / Disponível	%
Logic utilization (ALMs)	5.324 / 32.070	17 %
Registradores	8.622	—
Block memory bits	1.985.569 / 4.065.280	49 %
RAM Blocks	250 / 397	63 %
DSP Blocks	12 / 87	14 %
PLLs	2 / 6	33 %
Pinos I/O	382 / 457	84 %

O aumento de ALMs, registradores e memória em relação à versão antiga somente FPGA (porta_estimulo) deve-se à integração do HPS (soc_system), interface PIO, FIFOs de comando e ao núcleo ISA completo.

1.2 Distribuição aproximada por bloco

Módulo	Papel	Observação de custo
HPS + Qsys (soc_system)	Processador ARM + bridges	Maior parte dos pinos e memória
motor_sprites / rasterizador	32 sprites + polígonos	Principal uso de DSP e lógica gráfica
control_unit + isa_core	Decodificação ISA + FSM	Custo moderado de ALMs
RAMs (tilemap, padrões, FB)	Memórias de vídeo	Grande parte dos block RAM bits
pio_cmd_iface + fifo_cmd	Interface HPS ↔ núcleo	Baixo custo, essencial para SW

2. Timing e Fmax

No relatório de Timing Analyzer (Slow 1100 mV 85 °C), todos os clocks principais apresentam slack de setup positivo:

- clk_pix (meu_pll): Slack ≈ +2,81 ns
- clk ~100 MHz (pllpara100): Slack ≈ +1,52 ns
- clock_50: Slack ≈ +3,11 ns
- Hold positivo em todos os domínios reportados.

Diferentemente da versão anterior (porta_estimulo) em que o clk_pix apresentava slack negativo elevado, a versão HPS fecha timing nos domínios de pixel e de sistema com as constraints aplicadas.

3. Desempenho

- Resolução lógica 320×240 com expansão 2×2 para VGA reduz a pressão de banda de memória.
- Pipeline gráfico (background + sprites + polígonos + compositor) opera no domínio do pixel clock.
- Interface de comandos via FIFO + PIO permite enfileiramento de instruções ISA a partir do HPS sem travar o processador ARM.
- Double-buffering independente de tilemap (SWAP_BG) e frame-buffer (SWAP_FB) elimina tearing.

4. Gargalos

4.1 Caminho combinacional em clk_pix

Seleção de prioridade entre até 32 sprites, rasterização de polígonos e composição final ainda constituem o caminho mais longo no domínio de vídeo. O slack positivo atual indica que o design está dentro da margem, mas evoluções (mais sprites) devem monitorar este caminho.

4.2 Memória e banda

Leituras de tilemap, padrões e sprites competem no domínio do pixel. O uso de 49 % dos block memory bits e 63 % dos RAM blocks deixa margem, porém RAMs single-port limitam amostragens simultâneas.

4.3 Integração HPS

O tráfego PIO é leve (palavras de 32 bits de instrução). O gargalo de sistema, se existir, estará no software (frequência de envio de comandos) e não no hardware da interface.

5. Limitações

- Prioridade de camada em três níveis (BG / sprites / polígonos) e entre sprites por atributo (SPR_CFG).
- Transparência binária (cor 0);
- Padrões de sprite/tile em memórias internas (sem DMA direto da SDRAM nesta versão).
- Interface de comandos via PIO, suficiente para a ISA, mas não para streaming de framebuffers grandes.

6. Melhorias possíveis

6.1 Pipeline e timing

- Registrar saídas do rasterizador/compositor se o caminho de clk_pix apertar em evoluções futuras.
- Dividir a seleção de 32 sprites em estágios pipelineados se necessário.

6.2 Funcionalidade gráfica

- Paleta de 256 cores (RRRGGGBB) entre compositor e VGA.
- Mais slots de polígono ou modos adicionais (círculo, linhas) se o orçamento de DSP permitir.
- Páginas de memória ou DMA a partir da SDRAM para padrões maiores.

6.3 Integração de sistema

- Driver Linux + biblioteca C já existentes (coproc_driver); evoluir para API de jogo de mais alto nível.